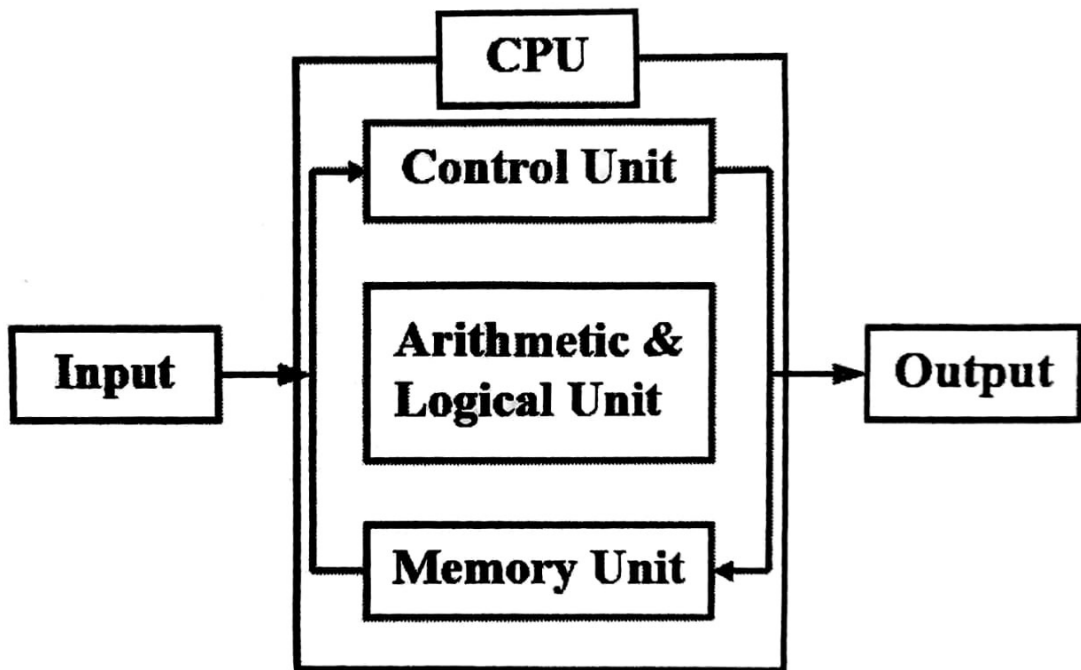


Component of a microcomputer system



09/28/2026

Md. Adnan Sami, Dept. of CSE, VU

5

Component of a microcomputer system

Working Principal:

1. **Central processing unit (CPU):** It controls the operation of the computer.
2. **Memory:** It stores data.
3. **I/O Device:** Moves data between the computer and its external environment.

वैश्व
विश्वविद्यालय

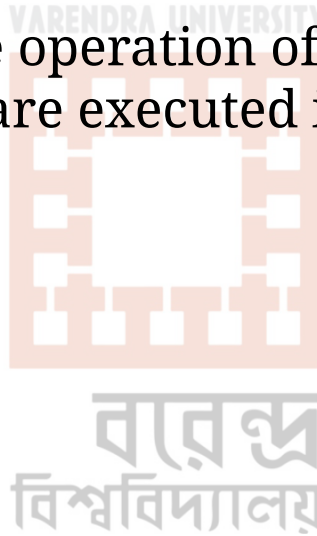
09/28/2026

Md. Adnan Sami, Dept. of CSE, VU

6

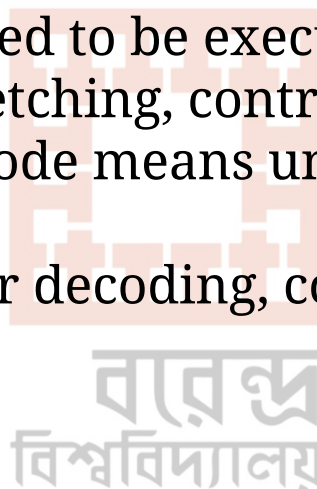
Why CPU is called the brain of the computer

- **CPU** controls the operation of the computer.
- All instructions are executed in CPU.

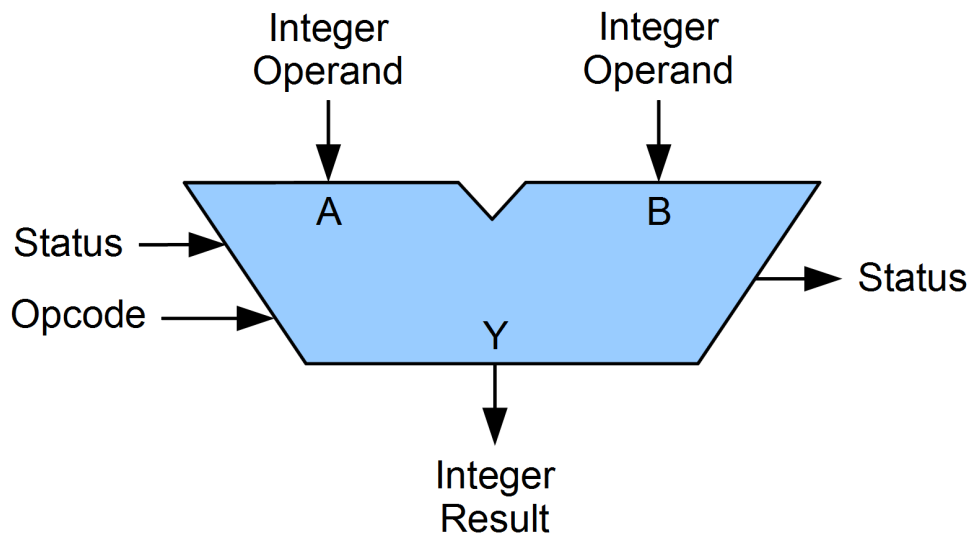


Operation or Function of Control Unit

1. **Fetch:** Control unit fetches instructions from memory that need to be executed.
2. **Decode:** After fetching, control unit decodes the instruction. Decode means understand the binary pattern.
3. **Execution:** After decoding, control unit executes the instruction.



Operation of Arithmetic Logic Unit (ALU)



09/28/2026

Md. Adnan Sami, Dept. of CSE, VU

9

Operation of Arithmetic Logic Unit (ALU)

- 1. Arithmetic Operation:** Arithmetic operation like addition, subtraction, multiplication etc. is performed.
- 2. Logical Operation:** Logical operation like AND, OR, NOT is performed.
- 3. Data Manipulation:** Altering or transforming data to make it more manageable and meaningful.

09/28/2026

Md. Adnan Sami, Dept. of CSE, VU

10

Microprocessor

Microprocessor is a processor which incorporates the functions of a CPU on a single integrated circuit (IC).

Types of microprocessor:

1. 8086
2. 8085
3. 8088



Application of microprocessor

1. Personal computer.
2. Mobile device.

Microcontroller

Microcontroller is a compact integrated circuit designed to govern a specific operation in an embedded system.

Embedded systems are focused on controlling and monitoring specific functions within a device.



Application of microcontroller:

1. Washing Machine.
2. Fire detection and safety device.

Difference between microprocessor and microcontroller

Microprocessor	Microcontroller
It is used for big application.	It is used to execute a single task.
It is the heart of computer system.	It is the heart of the embedded system.
It is just a processor. Memory and I/O components have to be connected externally.	It contains external processor along with internal memory and I/O components.
Power consumption is high.	Power consumption is low.
Cost is more.	Cost is less.
Difficult to replace.	Easy to replace.
Application: Personal computer.	Application: Washing machine.

09/28/2026

Md. Adnan Sami, Dept. of CSE, VU

5

8080/8085 Microprocessor

- 8 bit microprocessor.
- Developed by INTEL in the mid 1970.
- NMOS technology.
- Address bus 16 bit. ($2^{16} = 65,536$ byte or 64 KB)
($65,536/1024 = 64$)
- Data bus 8 bit.
- **Non-pipelined processor.**
- It has 3 available clock speeds (3 MHz, 5 MHz, and 6 MHz).
- 8085 has memory capacity of 64 KB.
- It does not have an instruction queue.

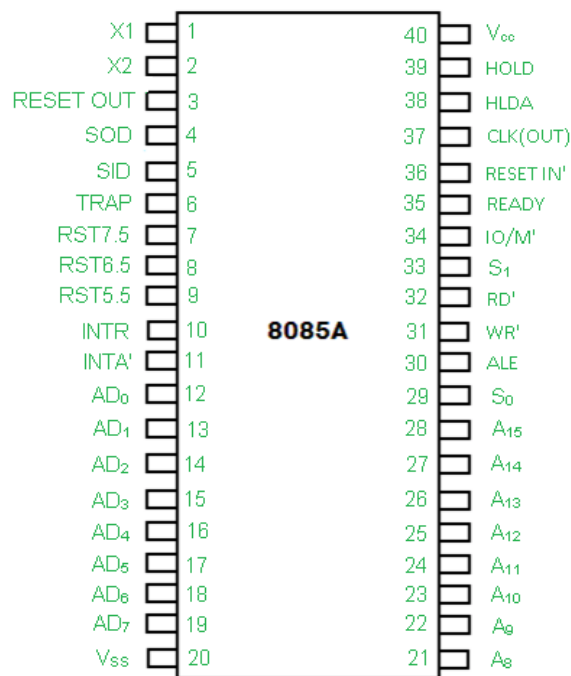
09/28/2026

Md. Adnan Sami, Dept. of CSE, VU

9

8080/8085 Microprocessor

Pin diagram of 8085 microprocessor.

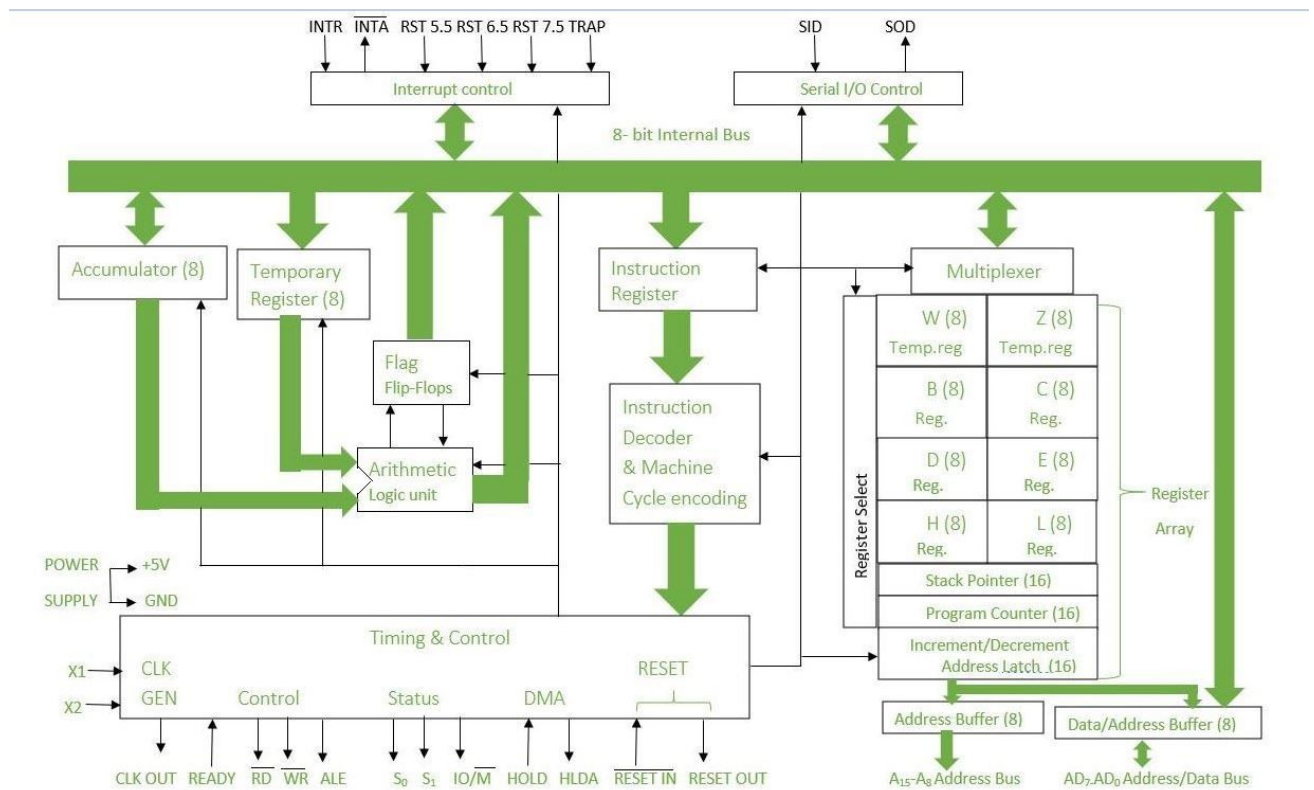


09/28/2026

Md. Adnan Sami, Dept. of CSE, VU

10

8080/8085 Microprocessor



09/28/2026

Md. Adnan Sami, Dept. of CSE, VU

11

8086 Microprocessor

- 16 bit microprocessor.
- Developed by INTEL in the year 1978.
- Originally HMOS, now manufactured using HMOS III technique.
- Approximately 29, 000 transistors and 40 pin.
- 5V supply.
- 33% duty cycle.
- Address bus 20 bit. ($2^{20} = 1 \text{ MB}$)
- Data bus 16 bit.
- **Pipelined processor.**
- It has 3 available clock speeds (5 MHz, 8 MHz and 10 MHz).
- 8086 has memory capacity of 1 MB.
- It has 6 byte instruction queue.

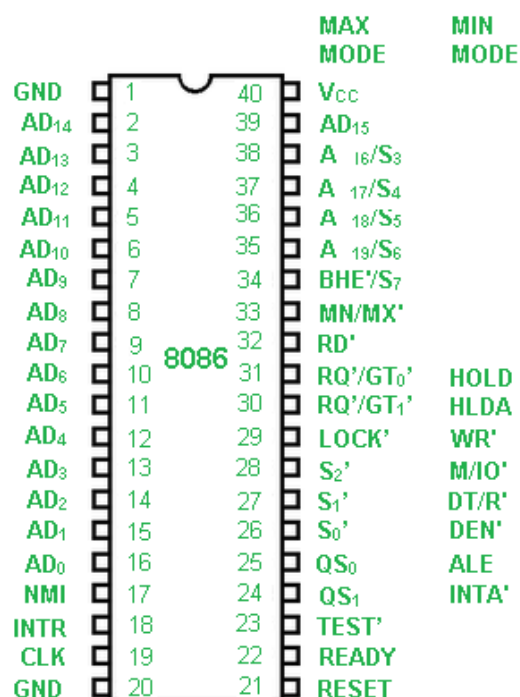
09/28/2026

Md. Adnan Sami, Dept. of CSE, VU

12

8086 Microprocessor

Pin diagram of 8086 microprocessor.



09/28/2026

Md. Adnan Sami, Dept. of CSE, VU

13

Difference between 8085 and 8086

Microprocessor

8085	8086
8 bit microprocessor.	16 bit microprocessor.
Address bus 16 bit.	Address bus 20 bit.
Data bus 8 bit.	Data bus 16 bit.
Non-pipelined processor.	Pipelined processor.
Memory capacity 64 KB.	Memory capacity 1 MB.
NMOS technology.	HMOS III technology.

विश्वविद्यालय

09/28/2026

Md. Adnan Sami, Dept. of CSE, VU

14

Pipelining

The process of fetching the next instruction when the present instruction is being executed is called as pipelining.

Non-pipelined:

Fetch 1	Execute 1	Fetch 2	Execute 2	Fetch 3	Execute 3
---------	-----------	---------	-----------	---------	-----------

Pipelined:

Fetch 1	Execute 1		
	Fetch 2	Execute 2	
		Fetch 3	Execute 3

09/28/2026

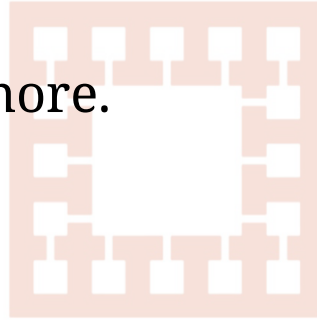
Md. Adnan Sami, Dept. of CSE, VU

15

Pipelining

Advantage of pipelining:

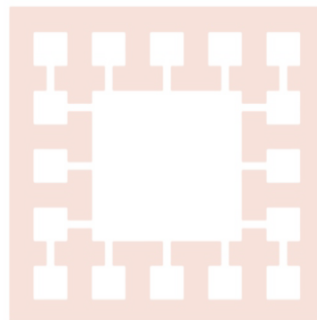
- Requires less time.
- Speed fast.
- Efficiency is more.



বরেন্দ্র
বিশ্ববিদ্যালয়

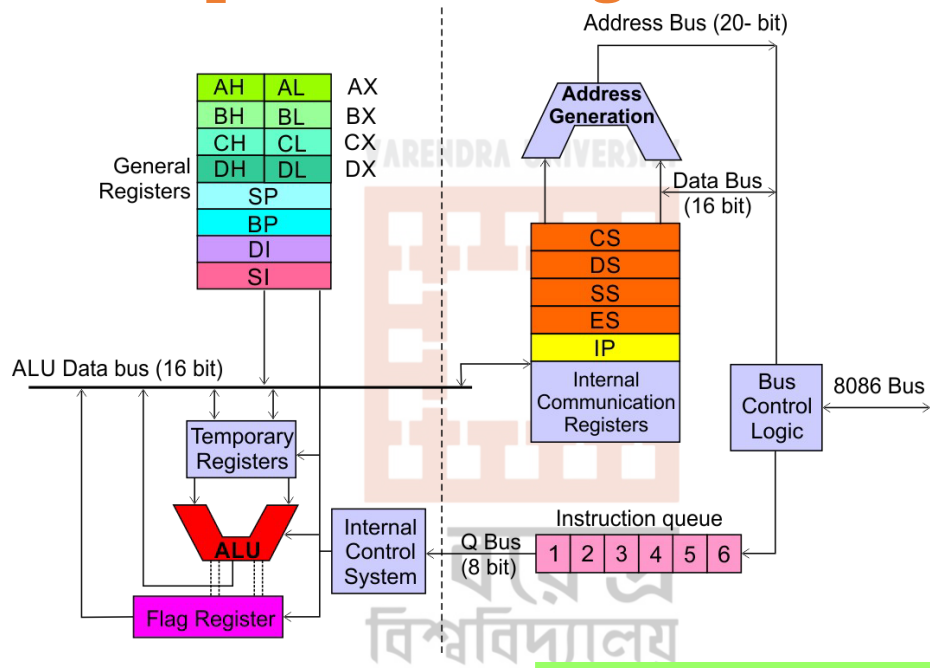
What is 16-bit processor

A processor that can process 16-bit data at a time is called a 16-bit processor.



বরেন্দ্র
বিশ্ববিদ্যালয়

8086 Microprocessor Organization



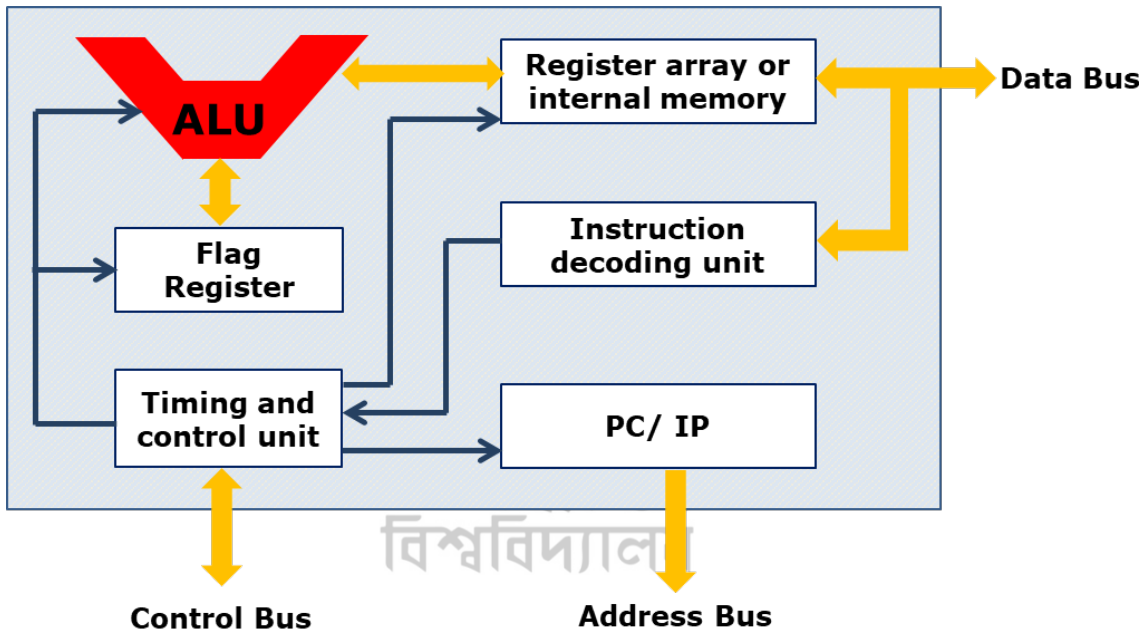
Execution Unit (EU)

- Instruction decode and execute.
- Arithmetic and logical operation.

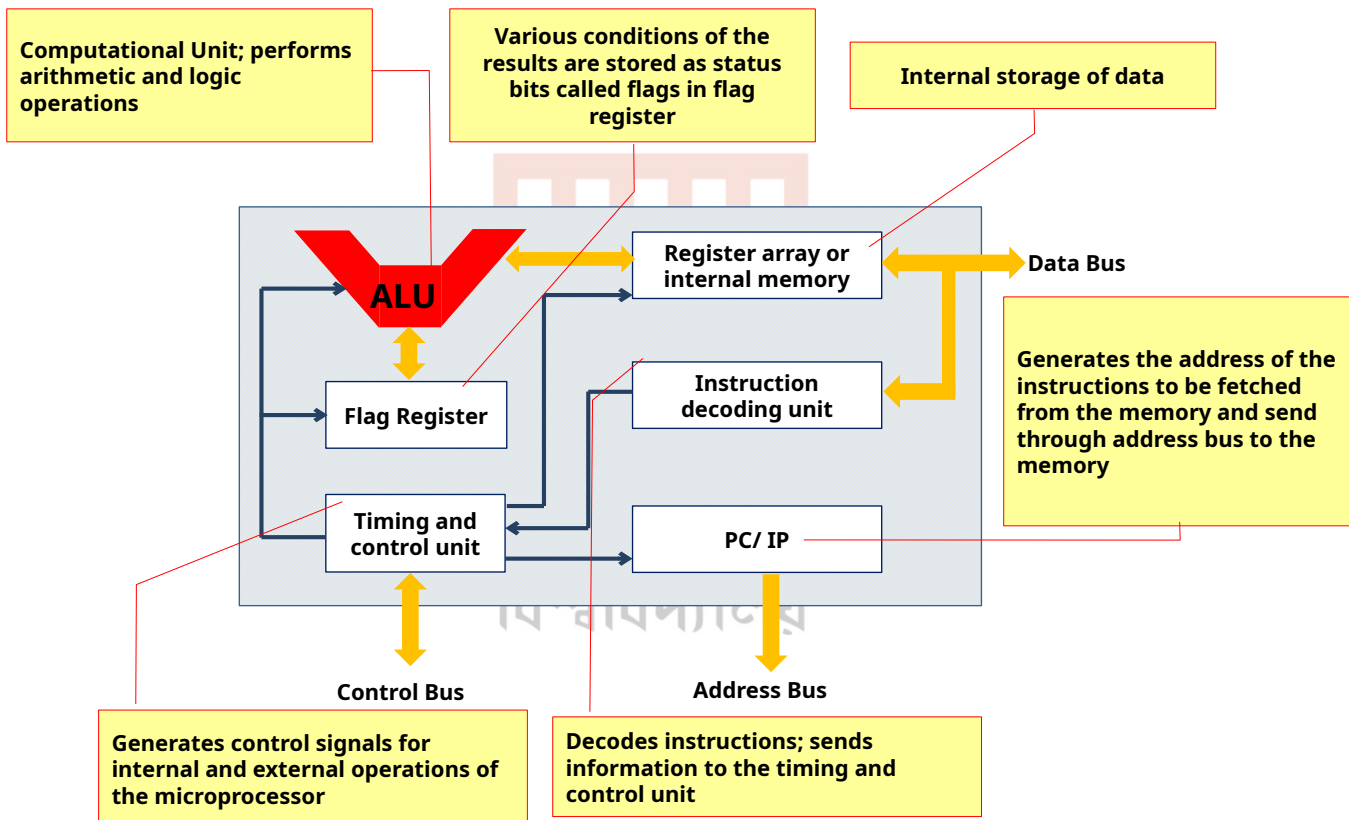
Bus Interface Unit (BIU)

- Instruction fetch.
- Address generation.
- Memory and I/O communication.

Functional Block of 8086 Microprocessor



Functional Block of 8086 Microprocessor



09/28/2026

Md. Adnan Sami, Dept. of CSE, VU

5

How does queue speed up the processing of the 8086 microprocessor?

The 8086 microprocessor's 6-byte instruction queue increases speed through instruction prefetching. The BIU fetches instructions and stores them in the queue. At the same time, the EU executes the instructions. Because of instruction fetching and execution overlap, the processor's idle time is reduced and execution speed increases.

09/28/2026

Md. Adnan Sami, Dept. of CSE, VU

6

Register of 8086 Microprocessor

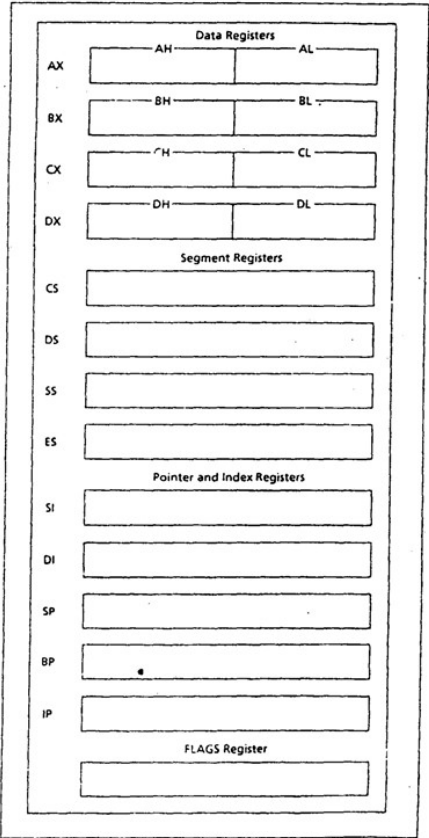
Category	Bits	Register Name
Data / General Purpose Registers	16	AX, BX, CX, DX
	8	AH, AL; BH, BL; CH, CL; DH, DL
Pointer	16	SP (Stack Pointer), BP (Base Pointer)
Index	16	SI (Source Index), DI (Destination Index)
Segment	16	CS (Code Segment) DS (Data Segment) SS (Stack Segment) ES (Extra Segment)
Instruction	16	IP (Instruction Pointer)
Flag	16	FR (Flag Register)

09/28/2026

Md. Adnan Sami, Dept. of CSE, VU

7

Register of 8086 Microprocessor



09/28/2026

Md. Adnan Sami, Dept. of CSE, VU

8

Register of 8086 Microprocessor

Data Register: Manipulates data.

- **AX (Accumulator Register):** Used in arithmetic, logic and data transfer instructions because it generates the shortest machine code.
- **BX (Base Register):** It is mainly address register. It contains data pointer which is base index. It indirectly addresses the register.
- **CX (Count Register):**
 - ❖ Program loop construction.
 - ❖ String manipulation.
 - ❖ Acts as a counter in shift or rotate instruction.
- **DX (Data Register):**
 - ❖ Works as the port number during the input and output operation.
 - ❖ Used in multiplication and division.

09/28/2026

Md. Adnan Sami, Dept. of CSE, VU

9

Register of 8086 Microprocessor

Segment Register: Stores address of instructions and data.

- **CS (Code Segment):** Used for storing instructions.
- **DS (Data Segment):** Used for storing data.
- **SS (Stack Segment):** Used as stack and store returning address.
- **ES (Extra Segment):** Used for storing destination data.

09/28/2026

Md. Adnan Sami, Dept. of CSE, VU

10

Register of 8086 Microprocessor

Pointer and Index Register: Points to the memory location. It is used in arithmetic and other operations.

- **SP (Stack Pointer):** Used for accessing the stack segment (SS).
- **BP (Base Pointer):** Used primarily to access data on the stack.
- **SI (Source Index):** Used to point memory locations in the data segment addressed by DS.
- **DI (Destination Index):** Performs the same functions as SI.

Register of 8086 Microprocessor

Flag Register: To indicate the status of the microprocessor. It contains the status and control flag.

- **Status flag:** Reflects the result of an instruction executed by the CPU.
- **Control flag:** Used to enable or disable certain operation of microprocessor.

Instruction set of 8086 microprocessor.

Data Transfer Instructions:

Instruction	Description
MOV	Transfer data between registers or between register and memory.
PUSH	Push register/memory data onto the stack.
POP	Pop data from stack into register/memory.
XCHG	Exchange data between two registers/memory.
IN	Input from a port to register.
OUT	Output from register to a port.
LEA	Load effective address of memory into a register.

09/28/2026

Md. Adnan Sami, Dept. of CSE, VU

13

Instruction set of 8086 microprocessor.

Arithmetic Instructions:

Instruction	Description
ADD	Add two values.
ADC	Add with carry.
SUB	Subtract one value from another.
SBB	Subtract with borrow.
MUL	Unsigned multiplication.
IMUL	Signed multiplication.
DIV	Unsigned division.
IDIV	Signed division.
INC	Increment value by 1.
DEC	Decrement value by 1.
NEG	Negate (2's complement).
CMP	Compare two values (subtract but don't store)

09/28/2026

Md. Adnan Sami, Dept. of CSE, VU

14

Instruction set of 8086 microprocessor.

Logical Instructions:

Instruction	Description
AND	Logical AND.
OR	Logical OR.
XOR	Logical Exclusive OR.
NOT	Logical Complement.
TEST	Logical AND (Result not stored).

Instruction set of 8086 microprocessor.

Shift and Rotate Instructions:

Instruction	Description
SHL	Shift bits left.
SHR	Shift bits right.
ROL	Rotate bits left.
ROR	Rotate bits right.
RCL	Rotate through carry left.
RCR	Rotate through carry right.

Instruction set of 8086 microprocessor.

Branching Instructions (Control Transfer

Instruction	Description
JMP	Unconditional jump.
RET	Return from a subroutine.
JE / JZ	Jump if equal (zero flag set).
JNE / JNZ	Jump if not equal (zero flag clear).
JG	Jump if greater.
JL	Jump if less.
JGE	Jump if greater or equal.
JLE	Jump if less or equal.
LOOP	Loop with CX register.
INT	Interrupt execution.

Instruction set of 8086 microprocessor.

String Manipulation Instructions:

Instruction	Description
MOVS	Move string from one location to another.
LODS	Load string element into AL/AX.
STOS	Store string element from AL/AX.
SCAS	Scan a string for a value.
CMPS	Compare two strings.
REP	Repeat instruction while CX $\neq$ 0.

Instruction set of 8086 microprocessor.

Processor Control Instructions:

Instruction	Description
STC	Set Carry flag.
CLC	Clear Carry flag.
CMC	Complement Carry flag.
STD	Set Direction flag.
CLD	Clear Direction flag.
STI	Set Interrupt flag (enable interrupts).
CLI	Clear Interrupt flag (disable interrupts).
HLT	Halt processor execution.
NOP	No operation (does nothing).
WAIT	Wait for processor signal.
ESC	Escape (for co-processor operations).
LOCK	Lock the bus to prevent other processors from

Instruction set of 8086 microprocessor.

The 8086 instruction set consists of:

- **Data Transfer Instructions:** Move data.
- **Arithmetic Instructions:** Perform calculations.
- **Logical Instructions:** Bitwise operations.
- **Shift and Rotate Instructions:** Bit manipulation.
- **Branching Instructions:** Control program flow.
- **String Instructions:** Process strings.
- **Processor Control Instructions:** Control CPU operations.

Assembly Language Syntax

Assembly language programs are translated into machine language instructions by an assembler.

Statements:

name	operation	operand(s)	comment
------	-----------	------------	---------

START:	MOV	CX, 5	;initialize counter
--------	-----	-------	---------------------

Here,

Name field: START:

Operation: MOV

Operand: CX, 5

Comment: ;initialize counter

Assembly Language Syntax

Statements:

name	operation
------	-----------

Here,

Name field: MAIN

Operation: PROC

Name Field

- The name field is used for instruction labels, procedure names, and variable names.
- The assembler does not differentiate between upper and lower case in a name.

Examples of legal names:

COUNTER1

@character

SUM_OF_DIGITS

\$1000

DONE?

.TEST

09/28/2026

Md. Adnan Sami, Dept. of CSE, VU

6

Name Field

Examples of illegal names

TWO WORDS

2abc

A45.28

YOU&ME

contains a blank

begins with a digit

. not first character

contains an illegal character

09/28/2026

Md. Adnan Sami, Dept. of CSE, VU

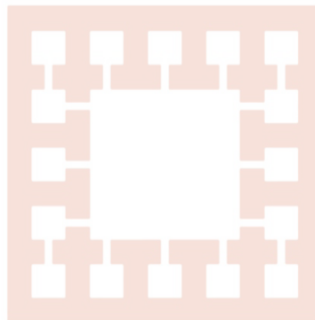
7

Operation Field

Operation's function:

- MOV
- ADD
- SUB

VARENDRA UNIVERSITY



বরেন্দ্র
বিশ্ববিদ্যালয়

Operand Field

Operand field specifies the data that are to be acted on by the operation.

VARENDRA UNIVERSITY



NOP	no operands, does nothing
INC AX	one operand; adds 1 to the contents of AX
ADD WORD1, 2	two operands; adds 2 to the contents of memory word WORD1

বরেন্দ্র
বিশ্ববিদ্যালয়

Comment Field

- The comment field of a statement is used by the programmer to say something about what the statement does.
- A semicolon marks the beginning of this field, and the assembler ignores anything typed after the semicolon.
- Comments are optional but assembly language is so low-level, it is almost impossible to understand an

```
MOV CX,0 ;move .0 to CX  
Instead, use comments to put the instruction into the context of the program:  
MOV CX,0 ;CX counts terms, initially 0
```

Program Data

- The processor operates only on binary data. The assembler must translate all data representation into binary numbers.
- An assembly language program we may express data as binary, decimal, or hex numbers, and even as characters.

Program Data

Numbers:

- A binary number is written as a bit string followed by the letter “B” or “b”. **For example:** 1010B.
- A decimal number is a string of decimal digits, ending with an optional “D” or “d”.
- A hex number must begin with a decimal digit and end with the letter “H” or “h”. **For example:** 0ABCH.

বাবু
বিশ্ববিদ্যালয়

Program Data

Number	Type
11011	decimal
11011B	binary
64223	decimal
-21843D	decimal
1,234	illegal—contains a nondigit character
1B4DH	hex
1B4D	illegal hex number—doesn't end in “H”
FFFFH	illegal hex number—doesn't begin with a decimal digit
0FFFFH	hex

Program Data

Characters:

- Characters and character strings must be enclosed in single or double quotes. **For example:** "A" or 'hello'

Table 4.1 Data-Defining Pseudo-ops

<i>Pseudo-op</i>	<i>Stands for</i>
DB	define byte
DW	define word
DD	define doubleword (two consecutive words)
DQ	define quadword (four consecutive words)
DT	define tenbytes (ten consecutive bytes)

Variable and Byte Variable

Variable:

- Variables play the same role in assembly language that they do in high-level languages.
- Each variable has a data type and is assigned a memory address by the program.

Byte Variable

Byte variable takes the following form:

```
name    DB          initial_value
where the pseudo-op DB stands for "Define Byte".
        For example,
ALPHA    DB          4
```

Variable and Byte Variable

Byte Variable

Byte variable is defined in the following form:

```
name DB initial_value
where the pseudo-op DB stands for "Define Byte".
For example,
ALPHA DB 4
```

Here,

name: ALPHA.

initial_value: 4.

Variable and Byte Variable

Byte Variable

A question mark (" ?") used in place of an initial value sets aside an uninitialized byte. For example:

```
BYT DB ?
```

Word Variable

Word variable is defined in the following form:

name	DW	initial_value
WRD	DW	-2

Here,

name: WRD.

initial_value: -2.

Arrays

In assembly language, an array is just a sequence of memory bytes or words.

For example:

To define a three-byte array called B_ARRAY, whose initial values are 10h, 20h, and 30h. We can write,

B_ARRAY	DB	10H, 20H, 30H
---------	----	---------------

Arrays

The name B_ARRAY is associated with the first of these bytes, B_ARRAY+1 with the second, and B_ARRAY+2 with the third. If the assembler assigns the offset address 0200h to B_ARRAY, then memory would look like this:

Symbol	Address	Contents
B_ARRAY	200h	10h
B_ARRAY+1	201h	20h
B_ARRAY+2	202h	30h

Arrays

The name B_ARRAY is associated with the first of these bytes, B_ARRAY+1 with the second, and B_ARRAY+2 with the third. If the assembler assigns the offset address 0200h to B_ARRAY, then memory would look like

Symbol	Address	Contents
B_ARRAY	200h	10h
B_ARRAY+1	201h	20h
B_ARRAY+2	202h	30h

In the same way, an array of words may be defined. For example,

W_ARRAY	DW	1000, 40, 29887, 329
---------	----	----------------------

Arrays

Character Strings:

```
LETTERS      DB      'ABC'

is equivalent to

LETTERS      DB      41H, 42H, 43H
```

Inside a string, the assembler differentiates between upper and lower case. Thus, the string "abc" is translated into three bytes with values 61h, 62h, and 63h.

Named Constants

EQU (Equates)

To assign a name to a constant, we can use the **EQU** (equates) pseudo-op. The syntax is

```
name          EQU          constant
```

For example, the statement

```
LF            EQU          0AH
```

assigns the name LF to 0Ah, the ASCII code of the line feed character. The name LF may now be used in place of 0Ah anywhere in the program. Thus, the assembler translates the instructions

```
MOV DL, 0AH
```

and

```
MOV DL, LF
```

into the same machine instruction.

The symbol on the right of an EQU can also be a string. For example,

```
PROMPT EQU 'TYPE YOUR NAME'
```

Then instead of

```
MSG DB 'TYPE YOUR NAME'
```

we could say

```
MSG DB PROMPT
```

Note: no memory is allocated for EQU names.

A Few Basic Instructions

MOV and XCHG:

The MOV Instruction is used to transfer data between registers, between a register and a memory location, or to move a number directly into a register or memory location. The syntax is,

```
MOV destination, source
```

Example:

```
MOV AX, WORD1
```

The contents of register AX are replaced by the contents of memory location WORD1. The contents of WORD1 are unchanged.

09/28/2026

Md. Adnan Sami, Dept. of CSE, VU

24

A Few Basic Instructions

MOV and XCHG:

MOV AX, BX

AX gets what was previously in BX. But BX is unchanged.

Figure 4.2 XCHG AH,BL

Before		After	
1A	00	05	00
AH	AL	AH	AL
00	05	00	1A
BH	BL	BH	BL

09/28/2026

Md. Adnan Sami, Dept. of CSE, VU

25

A Few Basic Instructions

MOV and XCHG:

Table 4.2 Legal Combinations of Operands for MOV and XCHG

Source Operand	Destination Operand			
	General register	Segment register	Memory location	Constant
General register	yes	yes	yes	no
Segment register	yes	no	yes	no
Memory location	yes	yes	no	no
Constant	yes	no	yes	no

Source Operand	Destination Operand	
	General register	Memory location
General register	yes	yes
Memory location	yes	no

A Few Basic Instructions

MOV and XCHG:

XCHG (exchange) operation is used to exchange the contents of two registers, or a register and a memory location. The syntax is `XCHG destination, source`

An example is

```
XCHG AH, BL
```

This instruction swaps the contents of AH and BL, so that AH contains what was previously in BL and BL contains what was originally in AH.

A Few Basic Instructions

MOV and XCHG:

Another example is,

```
XCHG AX, WORD1
```

Swaps the contents of AX and memory location WORD1.

A Few Basic Instructions

Restrictions on MOV and XCHG:

There are a few restrictions on the use of MOV and XCHG.

MOV or XCHG between memory locations is not allowed. For example,

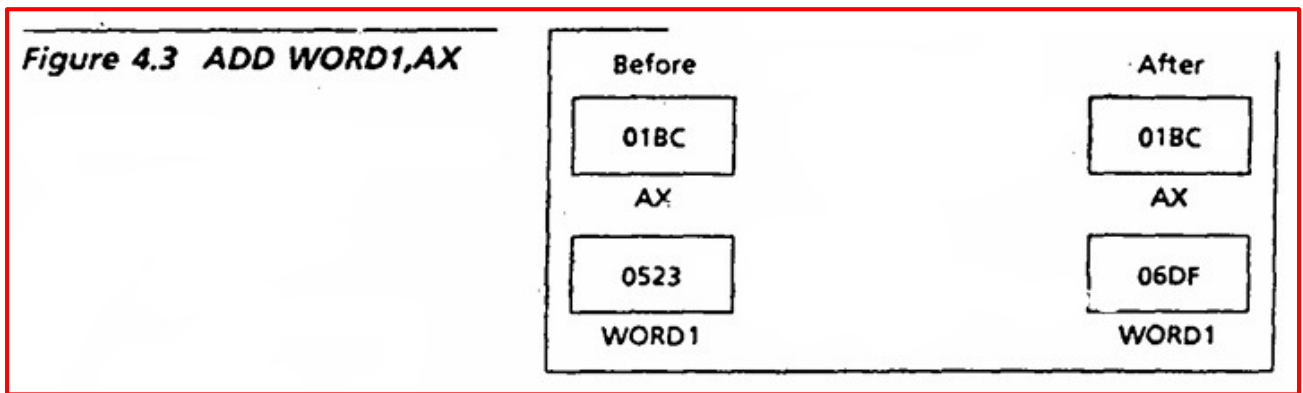
```
ILLEGAL: MOV WORD1, WORD2
```

Solution:

We can solve this by using a register.

```
MOV AX, WORD2  
MOV WORD1, AX
```

A Few Basic Instructions



A Few Basic Instructions

ADD, SUB, INC, DEC:

ADD and SUB instructions are used to add or subtract the contents of two registers, a register and a memory location. The syntax is,

```
ADD destination, source
SUB destination, source
```

Example:

i) `ADD WORD1, AX`

This instruction, "Add AX to WORD1" causes the contents of AX and memory word WORD1 to be added, and the sum is stored in WORD1. AX is unchanged.

A Few Basic Instructions

ADD, SUB, INC, DEC:

Example:

ii)

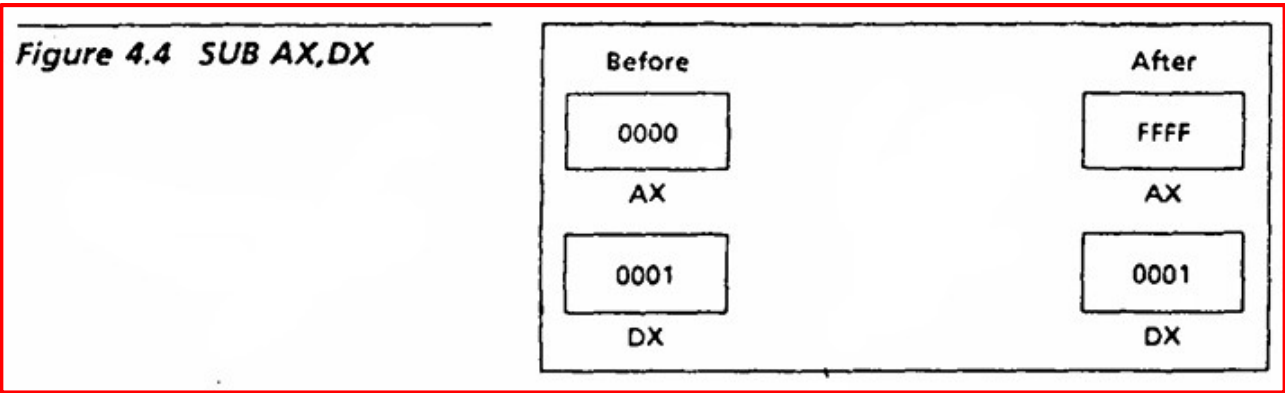
SUB AX, DX

This instruction, "**Subtract DX from AX**" the value of DX is subtracted from the value of AX, with the difference being stored in AX. DX is unchanged.

Table 4.3 Legal Combinations of Operands for ADD and SUB		
Source Operand	Destination Operand	
	General register	Memory location
General register	yes	yes
Memory location	yes	no
Constant	yes	yes

A Few Basic Instructions

ADD, SUB, INC, DEC:



A Few Basic Instructions

ADD, SUB, INC, DEC:

INC (increment) is used to add 1 to the contents of a register or memory location. The syntax is,

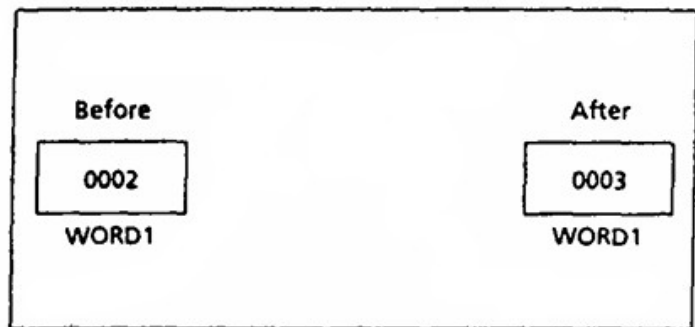
INC destination

Example:

INC WORD1

adds 1 to the contents of WORD1

Figure 4.5 INC WORD1



A Few Basic Instructions

ADD, SUB, INC, DEC:

DEC (decrement) is used to subtract 1 from a register or memory location. The syntax is,

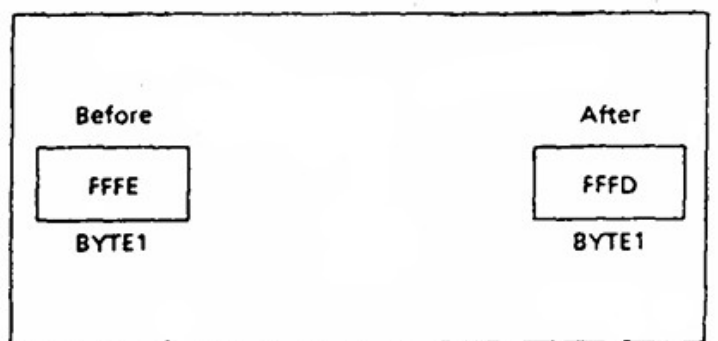
DEC destination

Example:

DEC BYTE1

subtracts 1 from variable BYTE1

Figure 4.6 DEC BYTE1



A Few Basic Instructions

ADD, SUB, INC, DEC:

NEG is used to negate the contents of the destination. NEG does this by replacing the contents by its two's complement. The syntax is,

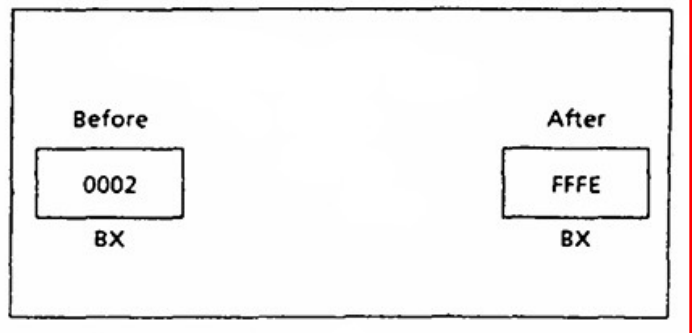
```
NEG destination
```

Example:

```
NEG BX
```

Negates the contents of BX.

Figure 4.7 NEG BX



09/28/2026

Md. Adnan Sami, Dept. of CSE, VU

36

Translation of High Level Language to Assembly Language

Statement

B = A

Translation

```
MOV AX, A ;move A into AX
MOV B, AX ;and then into B
```

As was pointed out earlier, a direct memory-memory move is illegal, so we must move the contents of A into a register before moving it to B.



09/28/2026

Md. Adnan Sami, Dept. of CSE, VU

37

Translation of High Level Language to Assembly Language

```
A = 5 - A      MOV AX, 5      ;put 5 in AX
                SUB AX, A      ;AX contains 5 - A
                MOV A, AX      ;put it in A
```

This example illustrates one approach to translating assignment statements: do the arithmetic in a register—for example, AX—then move the result into the destination variable. In this case, there is another, shorter way:

```
NEG A          ;A = -A
ADD A, 5        ;A = 5 - A
```

The next example shows how to do multiplication by a constant.

```
A = B - 2 x A   MOV AX, B      ;AX has B
                SUB AX, A      ;AX has B - A
                SUB AX, A      ;AX has B - 2 x A
                MOV A, AX      ;move result to A
```

Memory Models

The size of code and data a program can have is determined by specifying a memory model using the .MODEL directive. The syntax is,

`.MODEL``memory_model`

Table 4.4 Memory Models

Model	Description
SMALL	code in one segment data in one segment
MEDIUM	code in more than one segment data in one segment
COMPACT	code in one segment data in more than one segment
LARGE	code in more than one segment data in more than one segment no array larger than 64k bytes
HUGE	code in more than one segment data in more than one segment arrays may be larger than 64k bytes

Input and Output Instructions

There are two ways to do input and output on the IBM PC:

- 1) By direct communication with I/O devices.
- 2) By using BIOS or DOS routines.

BIOS (Basic Input Output System) routine: Stored in ROM and interact directly with the I/O ports.

DOS routine: Carry out more complex tasks. For example: Printing a character string.

CPU communicates with the peripherals through I/O registers called I/O ports.

Input and Output Instructions

Function number	Routine
1	single-key input
2	single-character output✓
9	character string output

A Program for Displaying String

```
.model small
.stack 100h

.data
a db 'Dept. of CSE $';Variable declaration

.code

main proc

    mov ax,@data ;Initialize the data segment to code segment
    mov ds,ax

    mov ah,9
    lea dx,a ;Load Effective Address
    int 21h

exit:
    mov ah,4ch
    int 21h
main endp
end main
```

09/28/2026

Md. Adnan Sami, Dept. of CSE, VU

42

A Program for Case Conversion

Lowercase to Uppercase:

VARENDRA U



বাবু বিশ্ববিদ্যালয়

```
.model small
.stack 100h
.code

main proc
    mov ah,1
    int 21h ;input
    mov bl,al

    sub bl,32

    mov ah,2
    mov dl,10
    int 21h
    mov dl,13
    int 21h

    mov ah,2
    mov dl,bl
    int 21h

exit:
    mov ah,4ch
    int 21h
main endp
end main
```

09/28/2026

Md. Adnan Sami, Dept. of CSE, VU

43

A Program for Case Conversion

Uppercase case to Lowercase:

```
.model small
.stack 100h
.code

main proc
    mov ah,1
    int 21h ;input
    mov bl,al

    add bl,32

    mov ah,2
    mov dl,10
    int 21h
    mov dl,13
    int 21h

    mov ah,2
    mov dl,bl
    int 21h

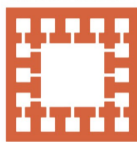
exit:
    mov ah,4ch
    int 21h
main endp
end main
```

09/28/2026

Md. Adnan Sami, Dept. of CSE, VU

44

VARENDRA UNIVERSITY



বরেন্দ্র
বিশ্ববিদ্যালয়

VARENDRA UNIVERSITY

Varendra University

Dept. of Computer Science & Engineering (CSE)

The Processor Status and the Flag Register (Chapter-5)

Course Title: **Microprocessor and Assembly Language**

Course No: **CSE 3205**

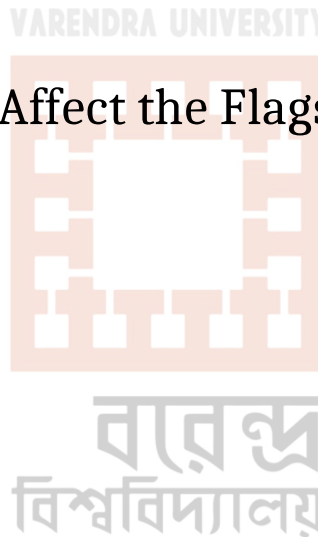
Prepared By:

Md. Adnan Sami

Lecturer, Dept. of CSE, VU

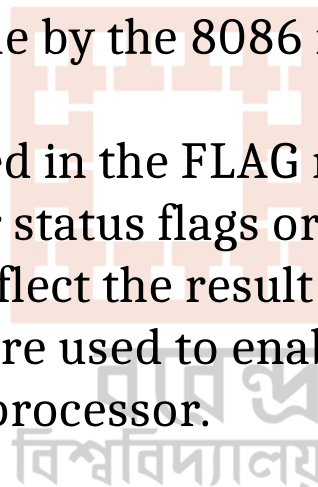
Outline

1. Flag Register.
2. Overflow.
3. How Instructions Affect the Flags.
4. Overflow Math.
5. Practice Problem.



Flag Register

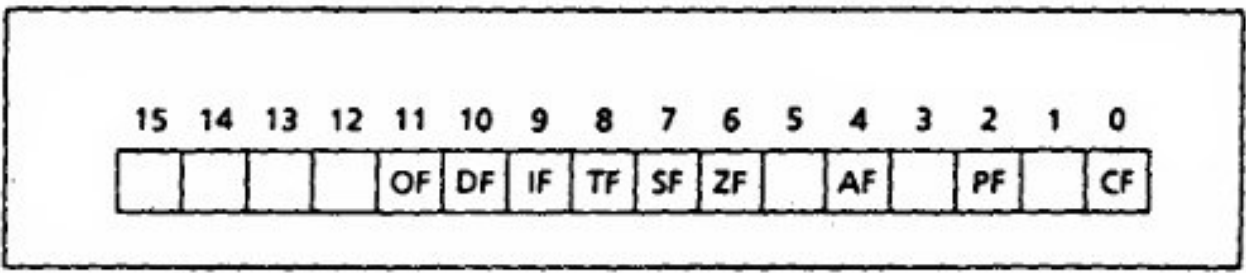
- For the 8086 processor, the processor state is implemented as nine individual bits called flags.
- Each decision made by the 8086 is based on the values of these flags.
- The flags are placed in the FLAG register and they are classified as either status flags or control flags.
- The status flags reflect the result of a computation.
- The control flags are used to enable or disable certain operations of the processor.



Flag Register

Flag register are two types,

- 1. Status Flag.
- 2. Control Flag.



- The status flags are located in bits 0, 2, 4, 6, 7, and 11.
- The control flags are located in bit 8, 9, and 10.

Flag Register

Table 5.1 Flag Names and Symbols		
Status Flags		
Bit	Name	Symbol
0	Carry flag	CF
2	Parity flag	PF
4	Auxiliary carry flag	AF
6	Zero flag	ZF
7	Sign flag	SF
11	Overflow flag	OF
Control Flags		
Bit	Name	Symbol
8	Trap flag	TF
9	Interrupt flag	IF
10	Direction flag	DF

Flag Register

Status Flag:

The processor uses status flags to reflect the result of an operation.

Carry Flag (CF):

- CF = 1, if there is a carry out from the most significant bit (MSB) on addition, or there is a borrow into the MSB on subtraction. Otherwise, it is 0.
- CF is also affected by shift and rotate instructions.

Flag Register

Parity Flag (PF):

- PF = 1, if the low byte of a result has an **even number of one** bits (even parity).
- It is 0 if the low byte has odd parity.

For example,

If the result of a word addition is FFFEh (**1111 1111 1111 1110**), then the low byte contains 7 one bits, so PF = 0.

Auxiliary Flag (AF):

- AF = 1, if there is a carry out from bit 3 on addition, or a borrow into bit 3 on subtraction.
- AF is used in binary-coded decimal (BCD) operations.

Flag Register

Zero Flag (ZF):

- $ZF = 1$ for a zero result.
- $ZF = 0$ for a nonzero result.

Sign Flag (SF):

- $SF = 1$, if the MSB of a result is 1 (It means the result is negative)
- $SF = 0$, if the MSB is 0.

Overflow Flag (OF):

- $OF = 1$, if signed overflow occurred, otherwise it is 0.

Overflow

Unsigned Overflow:

- On addition, unsigned overflow occurs when there is a carry out of the MSB.
- On subtraction, unsigned overflow occurs when there is a borrow into the MSB.

Overflow

Signed Overflow:

- On addition of numbers with the same sign, signed overflow occurs when the sum has a different sign. **For example**, when we add 7FFFh and 7FFFh (two positive numbers), but got FFFEh (a negative result).
- Subtraction of numbers with different signs is like adding numbers of the same sign. For example, $A - (-B) = A + B$ and $-A - (+B) = -A + -B$. Signed overflow occurs if the result has a different sign than expected.
- In addition of numbers with different signs, overflow is impossible.

8/28/2026

Md. Adnan Sami, Dept. of CSE, VU

10

How Instructions Affect the Flags

<i>Instruction</i>	<i>Affects flags</i>
MOV/XCHG	none
ADD/SUB	all
INC/DEC	all except CF
NEG	all (CF = 1 unless result is 0, OF = 1 if word operand is 8000h, or byte operand is 80h)

8/28/2026

Md. Adnan Sami, Dept. of CSE, VU

11

Overflow Math



8/28/2026

Md. Adnan Sami, Dept. of CSE, VU

12

Practice Problem

Example 5.1:

Example 5.1 ADD AX,BX, where AX contains FFFFh, BX contains FFFFh.

Solution:

$$\begin{array}{r} \text{FFFFh} \\ + \text{FFFFh} \\ \hline 1 \text{ FFEh} \end{array}$$

The result stored in AX is FFEh = 1111 1111 1111 1110.

SF = 1 because the msb is 1.

PF = 0 because there are 7 (odd number) of 1 bits in the low byte of the result.

ZF = 0 because the result is nonzero.

CF = 1 because there is a carry out of the msb on addition.

OF = 0 Because the sign of the stored result is the same as that of the numbers being added (as a binary addition, there is a carry into the msb and also a carry out).

8/28/2026

Md. Adnan Sami, Dept. of CSE, VU

13

Practice Problem

Example 5.2:

Example 5.2 ADD AL,BL, where AL contains 80h, BL contains 80h.

Solution:

$$\begin{array}{r} 80h \\ + 80h \\ \hline 100h \end{array}$$

The result stored in AL is 00h.

SF = 0 because the msb is 0.

PF = 1 because all the bits in the result are 0.

ZF = 1 because the result is 0.

CF = 1 because there is a carry out of the msb on addition.

OF = 1 because the numbers being added are both negative, but the result is 0 (as a binary addition, there is no carry into the msb but there is a carry out).

Practice Problem

Example 5.3:

Example 5.3 SUB AX,BX, where AX contains 8000h and BX contains 0001h.

Solution:

$$\begin{array}{r} 8000h \\ - 0001h \\ \hline 7FFFh = 0111\ 1111\ 1111\ 1111 \end{array}$$

The result stored in AX is 7FFFh.

SF = 0 because the msb is 0.

PF = 1 because there are 8 (even number) one bits in the low byte of the result.

ZF = 0 because the result is nonzero.

CF = 0 because a smaller unsigned number is being subtracted from a larger one.

Now for OF. In a signed sense, we are subtracting a positive number from a negative one, which is like adding two negatives. Because the result is positive (the wrong sign), OF = 1.

Practice Problem

Example 5.4:

Example 5.4 INC AL, where AL contains FFh.

Solution:

$$\begin{array}{r} \text{FFh} \\ + \text{1h} \\ \hline \text{100h} \end{array}$$

The result stored in AL is 00h. SF = 0, PF = 1, ZF = 1. Even though there is a carry out, CF is unaffected by INC. This means that if CF = 0 before the execution of the instruction, CF will still be 0 afterward.

OF = 0 because numbers of unlike sign are being added (there is a carry into the msb and also a carry out).

Practice Problem

Example 5.5:

Example 5.5 MOV AX, -5

Solution: The result stored in AX is -5 = FFFBh.

None of the flags are affected by MOV.



Practice Problem

Example 5.6:

Example 5.6 NEG AX, where AX contains 8000h.

Solution:

$$\begin{array}{r} 8000\text{h} = 1000\ 0000\ 0000\ 0000 \\ \text{one's complement} = 0111\ 1111\ 1111\ 1111 \\ \qquad \qquad \qquad \qquad \qquad \qquad + 1 \\ \hline 1000\ 0000\ 0000\ 0000 = 8000\text{h} \end{array}$$

The result stored in AX is 8000h.

SF = 1, PF = 1, ZF = 0.

CF = 1, because for NEG CF is always 1 unless the result is 0.

OF = 1, because the result is 8000h; when a number is negated, we would expect a sign change, but because 8000h is its own two's complement, there is no sign change.

In the next section, we introduce a program that lets us see the actual setting of the flags.